

Module 10 — Core Skills Drill: FastAPI Endpoint with Typed Retrieval

Due: Tonight, after the concept lecture. Resubmissions are accepted through Saturday of the assignment week.

This drill exercises the **typed-API (Application Programming Interface) boundary plus a single retrieval call** before the Lab adds Neo4j, the full RAG (Retrieval-Augmented Generation) composition, Next.js, and Docker on top. The scope is deliberately tight — the smallest scoped exercise that sets up Lab task 4 (the RAG wrap) directly.

You will wrap a pre-implemented retrieval function behind a `POST /retrieve` endpoint with Pydantic-typed request and response models, surface a graceful empty-retrieval case, and add a `GET /healthz` liveness probe.

Setup

If you have not already installed the Module 10 packages, go back to the [Setup: install before the concept session](#) section of the reading and complete the install before proceeding.

Then fork the drill template repo and clone it locally.

Drill template repo: [LevelUp-Applied-AI/m10-d10](#). Click “Use this template” → “Create a new repository” on the template page, name your copy `m10-d10`, leave Visibility as Public, and create the repository under your own GitHub account.

```
# After creating your copy from the template:
git clone https://github.com/<your-username>/m10-d10.git
cd m10-d10
git checkout -b drill-10-typed-retrieval-endpoint
python -m venv .venv
source .venv/bin/activate          # macOS / Linux
# .venv\Scripts\Activate.ps1      # Windows PowerShell
pip install -r requirements.txt
```

The starter ships:

- `main.py` — a FastAPI app skeleton with stub functions raising `NotImplementedError`.
- `retrieval.py` — a pre-implemented `retrieve_top_k(query: str, k: int = 3) -> list[dict]` function that scores a fixed in-memory fixture of 5 numbered chunks by simple token overlap. **Do not modify.**
- `requirements.txt` — `fastapi`, `uvicorn[standard]`, `pydantic`, `pytest`, `httpx` (`httpx` backs `fastapi.testclient.TestClient`, which the autograder uses to drive the app in-process — you do not call `httpx` directly).
- `tests/test_drill.py` — the autograder tests (read these as your acceptance criteria; you do not modify them).

There is no Weaviate, no Neo4j, no Docker in this drill — those are Lab and Integration concerns.

Task

Author `main.py` so that the autograder tests pass. Specifically:

- Define request and response models

```
from pydantic import BaseModel, Field

class RetrieveRequest(BaseModel):
    query: str = Field(..., min_length=1, max_length=500)
    k: int = Field(3, ge=1, le=10)

class Chunk(BaseModel):
    chunk_id: int
    text: str
    score: float

class RetrieveResponse(BaseModel):
    retrieved: list[Chunk]

class HealthResponse(BaseModel):
    status: str
```

Pydantic v2 idioms only. The `Field(..., min_length=1, max_length=500)` constraint on `query` and `Field(3, ge=1, le=10)` constraint on `k` are what the autograder uses to verify invalid input is rejected at the boundary.

- Wire the `POST /retrieve` endpoint

```
from fastapi import FastAPI
from retrieval import retrieve_top_k

app = FastAPI()

@app.post("/retrieve", response_model=RetrieveResponse)
def retrieve(req: RetrieveRequest) -> RetrieveResponse:
    raw = retrieve_top_k(req.query, req.k)
    # raw is list[{"chunk_id": int, "text": str, "score": float}]
    return RetrieveResponse(retrieved=[Chunk(**r) for r in raw])
```

The retrieval function is pre-implemented. **You do not implement retrieval — you wrap it.** That is the load-bearing rehearsal: a typed boundary around a function whose internals could be anything (in the Lab they will be a Weaviate `with_near_vector` call against externally-embedded chunks).

- Wire the `GET /healthz` endpoint

```
@app.get("/healthz", response_model=HealthResponse)
def healthz() -> HealthResponse:
    return HealthResponse(status="ok")
```

`/healthz` is process-level — it must not probe any backend. The Lab will add a separate `/readyz` that does probe backends; the drill only has `/healthz`.

- Handle the empty-retrieval case

If `retrieve_top_k` returns an empty list (the query had zero token overlap with any fixture chunk), the endpoint must return HTTP 200 with `{"retrieved": []}` — not 404, not 500. The starter's pre-implemented retrieval naturally returns `[]` for queries with no matches; your wrap should not need any special logic for this — returning `RetrieveResponse(retrieved=[])` from the same code path that handles the populated case is enough.

Running locally

```
uvicorn main:app --reload
```

Then in another terminal:

```
curl -X POST http://localhost:8000/retrieve \
  -H "Content-Type: application/json" \
  -d '{"query": "fastapi", "k": 2}'

curl http://localhost:8000/healthz
```

Open `http://localhost:8000/docs` in your browser to see the auto-generated Swagger UI (User Interface). Verify the request and response shapes match what you defined.

Run the autograder locally:

```
pytest tests/ -v
```

All 13 tests must pass.

Submission

- Commit your changes to the `drill-10-typed-retrieval-endpoint` branch.
- Push the branch.
- Open a Pull Request (PR) against the `main` branch of your fork.
- Confirm the autograder workflow run succeeds (the green check appears within ~1 minute).
- Copy the PR URL.

Your PR description must include:

- A one-sentence summary of what you built.
- Confirmation that `pytest tests/ -v` passes locally.
- Paste your PR URL into TalentLMS → Module 10 → Drill 10 to submit this assignment.

Common pitfalls

- Forgetting `min_length=1` on `query`.** An empty string is a valid `str`. Without the constraint, the boundary accepts `{"query": ""}` and the autograder's empty-input test fails.
- Forgetting `ge=1, le=10` on `k`.** Same pattern — without the constraint, `k=100` slips through.
- Returning 500 on zero matches.** The retrieval function returns `[]` for no-match queries; that should produce a 200 response with an empty list, not an error.
- Using Pydantic v1 idioms.** `.dict()`, `parse_obj`, `class Config:` are v1; this drill uses v2 (`model_dump()`, `model_validate()`, `model_config = ConfigDict(...)`).

